



TED UNIVERSITY

CMPE 492 – Senior Project II

Spring 2026

AKKE – Final Report

(SMART COMMAND AND CONTROL GLOVE)

Webpage: [İLTER-AKKE](#)

GitHub Repository: [AKKE-Smart-Command-Control-Glove](#)

Name Surname Department ID

Abdullah ESİN (CMPE) 14476636120

Berk ÇAKMAK (CMPE) 10948064620

Ömer Efe DİKİCİ (CMPE) 63058185488

Şevval KURTULMUŞ (EEE) 52654371036

Supervisors: Ali BERKOL and Hüseyin Uğur YILDIZ

Jury Members: Hakkı Gökhan İLK and Mehmet Evren COŞKUN

Table of Contents

1. Introduction.....	4
1.1 Purpose of the Document.....	4
1.2 Project Overview	4
1.3 Document Organization.....	5
2. Final System Architecture and Design	6
2.1 Architectural Evolution: From HLD to Final Implementation.....	6
2.2 System Overview	7
2.3 Glove Subsystem	7
2.3.1 Hardware Composition.....	7
2.3.2 Firmware State Machine.....	9
2.3.3 Calibration Procedure	9
2.3.4 Inference Pipeline	10
2.4 Receiver Subsystem.....	11
2.4.1 Hardware Composition.....	11
2.4.2 Audio Playback and Command Mapping.....	12
2.4.3 LCD User Interface, Volume Control, and Battery Monitoring.....	12
2.5 Communication Protocol	13
2.6 Confidence Gate	13
3. Machine Learning Pipeline.....	14
3.1 Data Collection Workflow.....	14
3.2 Dataset Structure.....	14
3.3 Feature Engineering.....	16
3.4 Deployed Model (Edge Impulse).....	17
3.5 Dataset Partitioning and Data Leakage Considerations.....	18
4. Project Status and Implementation	19
4.1 Status Summary	19
4.2 Reconciliation of Earlier Design Documents with Final Implementation.....	19
4.3 Items Deferred to Future Work.....	21
4.4 Environmental Robustness	21
5. New Tools and Technologies Used	22
5.1 Edge Impulse (TinyML Platform).....	22
5.2 ESP-NOW Protocol	22
5.3 Kalman Filtering for IMU Fusion.....	23
5.4 DFPlayer Mini MP3 Module	23
5.5 Other Tooling.....	23

5.6 Power Consumption and Runtime Characteristics	24
5.7 Memory Footprint and Embedded Resource Usage	25
6. Test Results	26
6.1 Test Approach	26
6.2 Functional Test Cases	26
6.3 Machine-Learning Model Performance	29
6.4 Performance Tests	31
6.5 Test Statistics and Assessment	31
7. Impact of Engineering Solutions	32
7.1 Global Context	32
7.2 Economic Context	32
7.3 Environmental Context	33
7.4 Societal Context	33
7.5 Ethical Considerations	34
8. Contemporary Issues Related to the Project	35
8.1 The Rise of TinyML and Edge AI	35
8.2 Wearable Computing for Tactical and Industrial Use	35
8.3 Silent and Low-Latency Communication Channels	36
8.4 Privacy, Bias, and Inclusivity in Gesture Recognition	36
9. Background Research and Resource Usage	37
9.1 Similar Smart-Glove Designs	37
9.2 Component Datasheets and Reference Materials	37
9.3 Engineering Principles and Software Methodology	38
9.4 Online Resources Consulted	38
10. Discussion and Future Work	39
10.1 Discussion	39
10.2 Future Work	40
11. Conclusion	41
12. Glossary / Acronyms	42
13. References	47
Appendix A: Implementation Repository Structure	48
Appendix B: Full Confusion Matrix	49

1. Introduction

1.1 Purpose of the Document

This Final Report documents the culmination of the AKKE (Smart Command and Control Glove) senior design project developed throughout the 2025–2026 academic year at TED University. The report consolidates the technical evolution of the system from the initial project proposal to the working hardware-software prototype that was demonstrated at the end of the spring semester. It refines and supersedes the design choices presented in the four preceding deliverables — the Project Specifications Report, the Analysis Report, the High-Level Design Report (HLD), and the Low-Level Design Report (LLD) — with a single coherent description of what was ultimately built, how it performs, and what challenges remain.

Following the requirements of the CMPE 492 syllabus, the report presents the final architecture and implementation, summarizes the test results obtained against the Test Plan, discusses the impact of the engineering solutions in global, economic, environmental, and societal contexts, addresses contemporary issues related to wearable command-and-control systems, lists the new tools and technologies adopted during the course, and documents the library and Internet resources consulted for background information.

1.2 Project Overview

AKKE is a wearable, real-time hand-gesture recognition system designed to enable silent, low-latency, and offline command transmission in tactical and high-noise environments where verbal communication is impractical or risky. The system consists of two physical units that communicate over a low-latency wireless link. The glove unit is fitted with five flex sensors that capture finger flexion, an MPU-6050 inertial measurement unit (IMU) that captures hand orientation and motion, an ESP32-WROOM-32 microcontroller that executes an embedded TinyML classifier locally, and a small set of physical controls (an RGB status LED and a calibration/inference button). When a gesture is recognized with sufficient confidence, the glove transmits a one-byte command identifier to a separate receiver unit over the ESP-NOW protocol. The receiver, itself built around an ESP32-WROOM-32, plays the corresponding pre-recorded audio command through a DFPlayer Mini module and a speaker, and displays the active command and current volume level on a 16×2 LCD screen.

At the end of the project, the deployed Edge Impulse classifier recognizes ten distinct tactical gestures — Stop, Listen, Come, Stick Together, Go-Go, Slow Down, Fall Back, Take Cover, Move Right, and Move Left. The int8-quantised model achieves 94.34% accuracy on the 813-sample held-out validation set, with weighted-average precision, recall, and F1-score values of approximately 0.94. All ten gestures remain above the original 75% recognition target, although Fall Back is the weakest class with an F1-score of 0.841. When the confidence gate is applied, ambiguous predictions are rejected as “uncertain” rather than transmitted, which reduces the displayed operational accuracy in Edge Impulse but improves safety by preventing low-confidence command transmission.

1.3 Document Organization

The remainder of this report is organized as follows. Section 2 presents the final system architecture and design, including the architectural revision that occurred between the HLD and the LLD, the hardware and firmware of both the glove and the receiver, and the communication protocol that binds them. Section 3 covers the machine-learning pipeline: data collection, dataset structure, feature engineering, the deployed Edge Impulse model, and the reference 1D-CNN implementation prepared in TensorFlow/Keras. Section 4 summarizes the final status of the project, including what was implemented, what was deferred, and the discrepancies between earlier design documents and the final code that are reconciled here. Section 5 lists the new tools and technologies adopted during the course. Section 6 reports the test results in accordance with the Test Plan. Sections 7 and 8 discuss the engineering impact and contemporary issues. Section 9 documents background resources. Sections 10–11 contain the discussion and conclusion, and Section 12 lists the references. Two appendices follow with the implementation repository structure and the full confusion matrix.

2. Final System Architecture and Design

2.1 Architectural Evolution: From HLD to Final Implementation

During the transition from the high-level design phase (CMPE 491, Fall 2025) to the low-level design and implementation phase (CMPE 492, Spring 2026), a substantial architectural revision was performed. The HLD proposed an analog audio broadcast architecture in which the glove would play a command audio file locally through a DFPlayer Mini, route the resulting analog signal to an RF/FM transmitter module, and broadcast it over the air to passive receiver radios. It also proposed an ESP32-S3 as the central microcontroller. Both choices were revised during implementation.

Microcontroller migration (ESP32-S3 → ESP32-WROOM-32). The central processing unit was changed to the ESP32-WROOM-32 module (built around the ESP32-D0WDQ6 chip) on both ends of the link. The decision was driven by four factors: immediate component availability in the Turkish market, lower unit cost, a more mature Arduino-core and Edge Impulse deployment toolchain for the WROOM-32 target (reducing TinyML integration risk), and sufficient computational headroom — the int8-quantised classifier runs well within the one-second responsiveness budget on the WROOM-32, making the additional vector instructions of the S3 unnecessary for this workload. The dual-core architecture and 520 KB SRAM of the WROOM-32 cover all real-time sensing, Kalman filtering, and inference tasks without contention.

Communication redesign (analog FM broadcast → ESP-NOW digital trigger). The analog broadcast pipeline was replaced with a digital-trigger architecture based on the ESP-NOW protocol: the glove transmits a one-byte command identifier and the receiver, which now contains its own ESP32-WROOM-32, looks up the corresponding audio file on its local microSD card and plays it through a DFPlayer Mini and a speaker. This shift reduced susceptibility to analog noise and interference, reduced communication latency to a few milliseconds per packet, lowered the radiated power footprint, and moved the heavy task of audio decoding away from the wearable unit. The trade-off is that the receiver is no longer a passive radio but an active device that must be paired in advance through its MAC address; the deployed pairing uses the address 08:A6:F7:47:5C:3C, hard-coded into the glove firmware.

The final architecture preserves the layered structure proposed in the HLD — hardware abstraction layer, data-processing layer, application layer — but anchors it on two cooperating ESP32 nodes rather than a single processor paired with a passive radio. All pin assignments, peripheral configurations, and timing budgets reported in subsequent sections refer to this deployed ESP32-WROOM-32 platform.

2.2 System Overview

AKKE is decomposed into two physical subsystems connected by a wireless control plane:

- The Glove Unit performs sensor acquisition, calibration, feature construction, on-device TinyML inference, confidence gating, and transmission of the recognized command.
- The Receiver Unit decodes incoming ESP-NOW packets, plays the corresponding audio file from microSD storage through a DFPlayer Mini and a speaker, and provides simple operator feedback on a 16×2 LCD.

Both subsystems run firmware written in C/C++ on top of the Arduino-ESP32 core. The wireless channel is the 2.4 GHz ISM band, used in connectionless ESP-NOW mode with a single-byte payload. There is no Wi-Fi association, no Bluetooth pairing, and no Internet connectivity: the system is intentionally air-gapped from external networks for operational security.

2.3 Glove Subsystem

2.3.1 Hardware Composition

The glove unit is built on a knitted glove substrate onto which five flex sensors are sewn along each finger. The sensor wires terminate at a small board cluster mounted on the back of the hand that contains the ESP32-WROOM-32 development board, the MPU-6050 IMU breakout, a battery-sense voltage-divider module, a step-down (buck) DC-DC converter, the 9 V alkaline battery, and a mechanical tactile switch that serves as the calibration/inference button. The RGB status LED is mounted in the same area for visibility while the glove is being worn. Table 2.1 lists the deployed pin map exactly as defined in `inference/inference.ino` in the project repository.

Power is provided by a 9 V alkaline battery connected to a buck converter with a regulated 5 V USB output. This output is supplied to the ESP32-WROOM-32 development board through its micro-USB connector. The ESP32 board's onboard regulator then generates the 3.3 V logic rail used by the microcontroller and sensors.

Function	ESP32-WROOM-32 GPIO	Notes
Flex sensor 1 (thumb)	GPIO 25	ADC1, 12-bit, 11 dB attenuation
Flex sensor 2 (index)	GPIO 33	ADC1 channel
Flex sensor 3 (middle)	GPIO 32	ADC1 channel
Flex sensor 4 (ring)	GPIO 35	ADC1 channel (input-only pin)
Flex sensor 5 (little)	GPIO 34	ADC1 channel (input-only pin)
MPU-6050 SDA	GPIO 21	I2C, 400 kHz, slave address 0x68
MPU-6050 SCL	GPIO 22	I2C, 400 kHz
Button (calibration / inference)	GPIO 13	INPUT_PULLUP, software debounces 200 ms
RGB LED – Red	GPIO 15	Digital, active-high
RGB LED – Green	GPIO 2	Digital, active-high
RGB LED – Blue	GPIO 0	Digital, active-high
Battery voltage sense	GPIO 39	Input-only ADC pin through voltage divider; used for battery-level monitoring and low-battery LED warning

Table 2.1. Pin assignments on the glove-side ESP32-WROOM-32.

The MPU-6050 is configured with a sample-rate divider of 7 (yielding an internal 1 kHz update rate), the digital low-pass filter disabled, a gyroscope full-scale range of ± 250 °/s, and an accelerometer full-scale range of ± 2 g. The wake-up and clock-source register (0x6B) is written with 0x01 to select the X-axis gyroscope reference oscillator. These values match the recommended low-noise configuration for human-motion sensing in the MPU-6050 datasheet.

A separate small voltage-divider module is wired to GPIO 39 (ADC1_CH3 / VN) to enable battery-level monitoring on the glove unit. The divider scales the raw battery voltage into the 0–3.3 V range accepted by the ESP32 ADC. The firmware periodically converts this ADC reading into an estimated battery percentage and uses the RGB LED to warn the user when the battery level is low.

2.3.2 Firmware State Machine

The glove firmware (inference/inference.ino) implements a four-state machine that governs the device's operating mode. The states are mutually exclusive, and transitions are driven by the calibration/inference button and by the completion of internal procedures.

State	LED Indication	Entry / Exit Behaviour
IDLE	Solid blue	Initial state after boot. Awaits the first button press; on press, transitions to CALIBRATING.
CALIBRATING	Yellow → Red → Magenta	Runs the three-stage calibration routine described in §2.3.3. On completion, transitions to READY.
READY	Solid green	Waiting for the user to perform a gesture. On button press, transitions to INFERRNCING.
INFERRNCING	Cyan during capture; Green or Red after decision	Captures a 100-sample window, runs the Edge Impulse classifier, applies the confidence gate, and either transmits the command (green) or rejects the decision (red, 1 s). Returns to READY.

Table 2.2. Glove firmware state machine.

2.3.3 Calibration Procedure

Calibration is critical because raw flex-sensor outputs vary substantially between sensors and between users due to fabric tension, sensor positioning, and inter-sensor manufacturing tolerances. The deployed routine runs three sequential stages, each lasting 2.5 seconds and announced to the user via the RGB LED:

- Open-hand calibration (yellow LED): the user holds the hand fully extended. The firmware averages twenty ADC samples per flex channel (sampled every 20 ms) to record the open-hand baseline `cal_open[i]`.
- Closed-fist calibration (red LED): the user closes the hand into a tight fist. Twenty samples per channel are averaged to record `cal_close[i]`.

- IMU stillness calibration (magenta LED): the user holds the hand still. The firmware reads the MPU-6050 fifty times (every 10 ms) and computes mean offsets for the three accelerometer axes (with a -16384 LSB correction on the Z axis to remove the 1 g gravity bias) and the three gyroscope axes. The Kalman filters for roll and pitch are then initialized from the first valid accelerometer-derived angles.

After all, three stages, the firmware emits the message `CALIBRATION COMPLETED` over the serial port and the LED turns solid green to indicate `READY`.

2.3.4 Inference Pipeline

Once in the `READY` state, a second button press triggers `performInference()`. This function collects exactly `EI_CLASSIFIER_RAW_SAMPLE_COUNT` samples (100 in the deployed model) at the sampling rate dictated by `EI_CLASSIFIER_FREQUENCY` (50 Hz, i.e. 20 ms per sample), giving a two-second capture window. The Arduino sketch uses the Edge-Impulse-provided constants directly rather than hard-coded literals, so the firmware automatically follows any future re-training that changes the window size or sampling rate without manual edits.

Each sample produces a thirteen-element feature vector:

- Five normalized flex values, mapped into $[0, 1]$ by $(\text{raw} - \text{cal_open}) / (\text{cal_close} - \text{cal_open})$, with clipping at the endpoints. If the difference between `cal_close` and `cal_open` is below one ADC count, the channel is forced to zero to avoid division by an unstable baseline.
- Two Kalman-filtered orientation angles — pitch and roll — computed from accelerometer-derived angles and gyroscope rates using the lightweight one-dimensional Kalman implementation from the `Kalman.h` library, with `dt` provided in seconds from a high-resolution `micros()` timer.
- Three offset-corrected accelerometer components (`acc_x`, `acc_y`, `acc_z`) in raw LSB units.
- Three offset-corrected gyroscope components (`gyro_x`, `gyro_y`, `gyro_z`) in raw LSB units.

The $100 \times 13 = 1300$ floats fill the `features[]` buffer. The buffer is wrapped in a `signal_t` structure via `numpy::signal_from_buffer()` and passed to `run_classifier()`. The classifier returns probabilities for each of the ten output classes. The label with the maximum probability is selected as the best candidate and its confidence value is compared to the 0.75 threshold (§2.6). If the threshold is met, `labelToCommand()` maps the textual label to a one-byte command ID in the range 1–10 and `sendCommandToReceiver()` transmits it over ESP-NOW. If not, the firmware blinks the LED red for one second and returns to `READY` without transmitting anything.

2.4 Receiver Subsystem

2.4.1 Hardware Composition

The receiver is housed in a small 3D-printed enclosure that is held in the hand. It contains a second ESP32-WROOM-32, a DFPlayer Mini MP3 module with its associated microSD card, a 16×2 I2C LCD screen (controller address 0x27), a low-profile speaker driven directly from the DFPlayer output, a panel potentiometer for volume adjustment, an LCD-contrast trimmer, a power switch, a USB-micro programming connector, a step-down (buck) DC-DC converter, and a 9 V alkaline primary battery. Table 2.3 lists the deployed pin map exactly as defined in `reciever.ino`.

The receiver unit uses the same power approach: a 9 V alkaline battery feeds a buck converter, whose regulated 5 V USB output powers the ESP32 development board through its micro-USB connector. The onboard regulator of the ESP32 board provides the 3.3 V logic rail for the controller-side electronics.

Function	ESP32-WROOM-32 GPIO	Notes
DFPlayer Mini RX (ESP32 → module)	GPIO 17 (TX1)	HardwareSerial 1, 9600 baud, 8N1
DFPlayer Mini TX (module → ESP32)	GPIO 16 (RX1)	HardwareSerial 1, 9600 baud, 8N1
Volume potentiometer	GPIO 36 (ADC1_CH0)	Polled every 120 ms; 0–4095 mapped to 0–30 volume
16×2 I2C LCD – SDA	GPIO 21	Default I2C, slave address 0x27
16×2 I2C LCD – SCL	GPIO 22	Default I2C
Battery voltage sense	GPIO 35	ADC1 input through voltage divider

Table 2.3. Pin assignments on the receiver-side ESP32-WROOM-32.

2.4.2 Audio Playback and Command Mapping

Pre-recorded MP3 files numbered 0001.mp3 through 0010.mp3 are stored in the /mp3/ directory of the microSD card. Each file contains a short-spoken rendition of the corresponding tactical command in the language and accent suitable for the operational scenario. On receiving a valid command identifier (1–10) the firmware calls `myDFPlayer.playMp3Folder(cmd)`, which selects the matching file by index. Receiving an out-of-range value triggers a serial warning and is discarded.

The canonical command-to-gesture mapping deployed in the receiver firmware is:

Command ID	Gesture Name	Audio Track
1	Stop	/mp3/0001.mp3
2	Listen	/mp3/0002.mp3
3	Go go	/mp3/0003.mp3
4	Come	/mp3/0004.mp3
5	Stick together	/mp3/0005.mp3
6	Slow down	/mp3/0006.mp3
7	Fall back	/mp3/0007.mp3
8	Take cover	/mp3/0008.mp3
9	Move right	/mp3/0009.mp3
10	Move left	/mp3/0010.mp3

Table 2.4. Final command mapping deployed in receiver.ino.

2.4.3 LCD User Interface, Volume Control, and Battery Monitoring

The 16×2 LCD displays a minimal two-line interface. The first line shows the human-readable label of the most recently played command (or “Wait...” when idle). The second line shows the current volume as Vol: N/30. The volume potentiometer is sampled every 120 ms; the raw ADC value is mapped to the 0–30 range expected by the DFPlayer, and the LCD is redrawn only when the value or the active command changes, to minimize I2C traffic and avoid visible flicker. The receiver firmware also reads the battery voltage through GPIO 35 using a voltage-divider input and converts it into an estimated battery percentage. This allows the receiver unit to monitor its own supply level during operation.

2.5 Communication Protocol

The two subsystems exchange data over Espressif's ESP-NOW protocol, a connectionless peer-to-peer link layer that operates on the 2.4 GHz ISM band and reuses Wi-Fi action-frame mechanisms defined under IEEE 802.11. ESP-NOW does not require Wi-Fi association or scanning, which makes it suitable for the low-latency, low-overhead command channel that AKKE needs. The payload is a single byte representing the command identifier, packed into a one-field struct: `typedef struct { uint8_t fileNumber; } SendData;`

On the glove side the receiver is registered as a peer at boot using its MAC address. The send callback `OnDataSent` reports success or failure of each transmission over the serial port. On the receiver side the `OnDataRecv` callback validates the payload size, verifies that the command identifier is within the 1–10 range, and atomically stores the value in a volatile `pendingCommand` variable. The main loop picks up the pending command, masks interrupts briefly while reading it, and dispatches the playback request to the `DFPlayer`. The combination of fixed-size payloads, explicit range validation, and atomic reads keeps the receiver robust against malformed packets and against the case where two transmissions arrive close together.

2.6 Confidence Gate

A confidence threshold of 0.75 is applied to every inference result before any command is transmitted. The threshold is hard coded in `inference/inference.ino` (line 416) as the float literal `0.75f` and gates the call to `sendCommandToReceiver()`. When `max_score ≥ 0.75` the LED turns solid green and the command is transmitted; when `max_score < 0.75` the LED blinks red for one second, the serial port prints the warning “below confidence threshold”, and the firmware returns to `READY` without transmitting. This decision gate is the principal safeguard against silent miss-classification: it ensures that ambiguous gestures simply produce no command rather than the wrong command, which is the safety-relevant failure mode in a tactical context.

3. Machine Learning Pipeline

3.1 Data Collection Workflow

The training data was collected with a separate Arduino sketch, `data_collection/data_collection.ino`, and a Python companion script, `data_collector.py`, running on the host computer over the USB serial link. The Arduino sketch uses the same pin map, calibration routine, and feature definition as the inference sketch, ensuring that the data the model is trained on matches the data the model sees at run time. The only structural difference is that the data-collection sketch captures a three-second window (rather than two) and tags each line with a DATA marker that the Python script parses and routes into a CSV file.

Each recording session followed the same procedure. The operator started the Python script and specified the target gesture label. The microcontroller LED indicated the current state of the system. After the three-stage calibration described in §2.3.3, the user pressed the button to begin a recording, performed the target gesture during the three-second window, and waited for the LED to turn green again before the next repetition. The Python script automatically incremented the `record_id` between repetitions and aggregated all rows into a single `dataset.csv` file.

3.2 Dataset Structure

The aggregated dataset (`data_collection/dataset.csv`) contains 725,550 rows of sensor data structured as follows:

Column	Type	Description
<code>record_id</code>	uint64	Identifier shared by all rows belonging to one three-second recording.
<code>timestamp_ms</code>	uint64	Millisecond timestamp at the moment the row was captured.
<code>label</code>	uint8 (1–10)	Numeric gesture label.
<code>flex_1 ... flex_5</code>	float (0.0–1.0)	Normalized flex-sensor values after per-user calibration.
<code>kalman_pitch</code>	float ($\pm 180^\circ$)	Kalman-filtered pitch angle from the IMU.
<code>kalman_roll</code>	float ($\pm 180^\circ$)	Kalman-filtered roll angle from the IMU.

acc_x, acc_y, acc_z	float	Offset-corrected accelerometer values in raw LSB.
gyro_x, gyro_y, gyro_z	float	Offset-corrected gyroscope values in raw LSB.

Table 3.1. Columns of the aggregated dataset.csv.

A second view of the same data is held in the `edge_impulse_data/` directory, where every recording is exported as a separate CSV file named `Hareket_X.<timestamp>.csv`. This is the form Edge Impulse expects for ingestion. The directory contains 4,837 individual recording files. Table 3.2 reports the per-class distribution as computed by `count_labels.py`.

Class (Internal Label)	Gesture	Recordings	Approx. Rows
Hareket_1	Stop	500	75,000
Hareket_2	Listen	725	108,750
Hareket_3	Go go	500	75,000
Hareket_4	Come	402	60,300
Hareket_5	Stick together	400	60,000
Hareket_6	Slow down	500	75,000
Hareket_7	Fall back	400	60,000
Hareket_8	Take cover	410	61,500
Hareket_9	Move right	500	75,000
Hareket_10	Move left	500	75,000
Total	10 classes	4,837	725,550

Table 3.2. Per-class dataset distribution.

The dataset is mildly imbalanced — Listen roughly 80% more recordings than Go-Go and Fall-Back — because Listen was the gesture used during the initial pipeline validation and more sessions were recorded for it than for later additions. The imbalance is small enough that no class-weighting was needed during training; the impact is visible only in the confidence figures of the rarer classes.

3.3 Feature Engineering

Two design choices in feature engineering deserve emphasis. First, the flex-sensor outputs are normalized on-device using per-session, per-user calibration. This is what makes the model portable across hands of different size and across rebuilds of the glove with slightly different flex-sensor tensions. Without this step, the model would have to learn a separate decision boundary for every hand it encounters, which is infeasible with the available data volume.

Second, the IMU stream is decomposed into two complementary representations. The Kalman-filtered pitch and roll angles supply a stable orientation estimate that is robust to vibration and that encodes the static pose of the hand at the end of the gesture. The raw, offset-corrected accelerometer and gyroscope channels supply the dynamic motion content that distinguishes, for example, an outward arm flick from a downward chop with similar end pose. Together the seven IMU channels capture both “where the hand ends up” and “how the hand got there.”

An early attempt to enrich the dataset with synthetically generated samples produced the opposite of the desired effect: validation accuracy dropped from approximately 60–70% on real data alone to roughly 25% when the synthetic samples were mixed in. Inspection showed that the synthetic samples assumed a sensor-noise distribution that did not match the real flex sensors, which exhibit substantial inter-sensor variability and non-Gaussian drift. The synthetic samples were dropped from the final pipeline. This is one of the most important practical lessons of the project: physical-sensor variability is hard to model statistically, and synthetic augmentation must preserve the actual noise process of the deployed hardware to be useful.

3.4 Deployed Model (Edge Impulse)

The model that runs on the glove was trained on Edge Impulse, exported as the Arduino library `ei-ilter-akke_1.3.3-arduino-1.0.3-impulse-#1.zip`, and integrated into the inference firmware via the generated header file `ILTER-AKKE_1.3.3_inferencing.h`. The pipeline in Edge Impulse consists of three blocks: a raw-data input block that consumes the thirteen-channel, 50 Hz time series; a window of two seconds with a 200 ms window increase (yielding 100 samples per window); and a classification block that combines a small Conv1D-based feature extractor with a softmax output over the ten classes. Because the 2 s window slides across each 3 s raw recording in 200 ms steps, Edge Impulse extracts six overlapping windows per recording, expanding the 4,837 raw recordings into approximately 29,022 training samples before the 80/20 train–test split is applied. Standardization parameters are computed from the training set, and the normalized time-series windows are then passed to a compact Conv1D-based classifier with a softmax output layer. The trained Keras model is quantized to int8 for deployment on the ESP32-WROOM-32. The training was performed with an 80/20 train–test split.

The compile-time constants defined in the exported header (`EI_CLASSIFIER_FREQUENCY`, `EI_CLASSIFIER_RAW_SAMPLES_PER_FRAME`, `EI_CLASSIFIER_RAW_SAMPLE_COUNT`, `EI_CLASSIFIER_LABEL_COUNT`, and `EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE`) are used directly by the inference sketch rather than hard-coded literals, which guarantees that the firmware tracks any future re-training. A serial banner prints these values at boot, allowing the operator to verify that the firmware on the device matches the model that was actually trained.

3.5 Dataset Partitioning and Data Leakage Considerations

An important methodological consideration in time-series machine-learning systems is the risk of dataset leakage caused by overlapping windows derived from the same raw recording.

In the AKKE pipeline, each raw recording captured during data collection is approximately three seconds long. During Edge Impulse preprocessing, these recordings are transformed into overlapping two-second windows using a sliding-window mechanism with a 200 ms window shift. Consequently, multiple highly correlated windows may originate from the same original recording.

The team recognizes that, depending on the internal train/test partitioning strategy used by Edge Impulse, overlapping windows originating from a single raw recording may potentially appear in both the training and validation subsets. This can artificially inflate reported accuracy because temporally adjacent windows share substantial sensor similarity.

Therefore, the reported validation accuracy of 94.34% should be interpreted as a controlled-development evaluation rather than a strict subject-independent generalization benchmark.

Several observations partially mitigate this concern:

- the dataset was collected across many independent recording sessions rather than a single continuous session,
- substantial natural variation exists in flex-sensor placement, hand speed, wrist orientation, and user execution,
- the deployed model was additionally validated through repeated real-time live demonstrations outside the Edge Impulse evaluation environment.

Nevertheless, the team explicitly acknowledges that a stricter evaluation methodology would strengthen the scientific rigor of the project. Future work should therefore include:

- recording-level train/test separation,
- subject-independent cross-validation,
- leave-one-user-out evaluation,
- fully isolated temporal-session partitioning,
- real-world field validation under operational conditions.

The project intentionally reports this limitation transparently because preventing optimistic evaluation bias is an important engineering and scientific responsibility in applied TinyML systems.

4. Project Status and Implementation

4.1 Status Summary

At the time of submission of this Final Report, the AKKE prototype is fully assembled and operational end-to-end. The glove successfully captures sensor data, runs on-device inference, applies the confidence gate, and transmits the selected command to the receiver over ESP-NOW. The receiver decodes the command, plays the corresponding audio track through its DFPlayer Mini and speaker, and updates the LCD display. All ten target gestures have been trained, validated, and exercised in live demonstrations. The system has been demonstrated at distances exceeding the ten-meter wireless range target stipulated in REQ-NF-014 of the Project Specifications Report. A separate User's Manual document (AKKE_User_Manual.pdf, 22 pages), containing operating instructions, calibration procedure, gesture vocabulary, troubleshooting guide, and developer installation instructions, accompanies this report.

4.2 Reconciliation of Earlier Design Documents with Final Implementation

Throughout the project lifecycle, the architecture evolved as constraints surfaced during implementation. Table 4.1 documents the principal points where the four earlier reports disagreed with one another or with the final code and records the value that prevails in the deployed system. These reconciliations are the authoritative description of AKKE as built.

Item	Earlier Reports	Deployed Implementation
Microcontroller	ESP32-S3 (HLD, Analysis, Specifications, Poster)	ESP32-WROOM-32 on both glove and receiver (LLD revision; confirmed by code in both .ino files)
Wireless link	Analog audio over FM/RF transmitter (HLD, Specifications)	Digital command trigger over ESP-NOW, 1-byte payload (LLD; confirmed by <code>sendCommandToReceiver()</code>)
Audio storage	MicroSD on glove (HLD deployment diagram)	MicroSD on receiver (LLD; confirmed by DFPlayer on receiver in <code>reciever.ino</code>)
Sampling rate	Implicitly high (HLD); 200 Hz (LLD §7)	50 Hz (confirmed by 20 ms loop in <code>data_collection.ino</code> and by <code>EI_CLASSIFIER_FREQUENCY = 50</code>)

Window size	Implicit (HLD); 2 s (LLD §7)	2 s capture window (100 samples × 20 ms; confirmed by EI_CLASSIFIER_RAW_SAMPLE_COUNT and SEQ_LEN = 100 in train_tinyml.py). Raw recordings during data collection are three seconds; Edge Impulse extracts six overlapping 100-sample (2 s) sub-windows per recording during training.
Number of gestures	“At least five” (Specs, Analysis, HLD); five (LLD command-mapping table)	Ten classes deployed (Hareket_1 ... Hareket_10) as confirmed by EI_CLASSIFIER_LABEL_COUNT, by the model evaluation JSON, and by the receiver command map
Gesture names	“Advance / Hold / Retreat / Cover / Confirm” (LLD)	Stop / Listen / Go go / Come / Stick together / Slow down / Fall back / Take cover / Move right / Move left
Receiver	Passive radio (HLD); active ESP32 (LLD)	Active ESP32-WROOM-32 with DFPlayer Mini at 9600 baud, 16×2 I2C LCD, and volume potentiometer
End-to-end latency	< 1 s (HLD G2; LLD)	Maintained: 2 s capture + EI inference (tens of ms) + ESP-NOW (single-digit ms) + DFPlayer trigger
Wireless range	≥ 10 m (REQ-NF-014, HLD G4)	≥ 10 m verified during live testing
Battery	Li-Po (HLD Glossary)	9 V alkaline (primary, zinc–manganese dioxide) cell on both glove and receiver; each subsystem has its own battery.
Confidence threshold	75–80% (Specs); 75% (LLD)	Hard-coded 0.75 in inference.ino

Table 4.1. Reconciliation of earlier design statements with the deployed implementation.

4.3 Items Deferred to Future Work

Five of the fifteen gestures originally listed in the data-collection plan (README.md in the repository) were not included in the deployed model. These are: Check the area, Spread out, Get down, The surroundings are clean (Y-shaped hand sign), and The operation is over / gather in the vehicle (thumbs up). They were deferred because the time budget for the spring semester was consumed by hardware iteration and by the data quality issues described in §3.3. Adding them does not require any structural change to the firmware: the deployed code path transparently supports up to `EI_CLASSIFIER_LABEL_COUNT` classes, so the only steps needed are data collection, re-training in Edge Impulse, regeneration of the inference library, and an addition of the new entries to the command-to-text and command-to-audio maps in the two firmware files.

4.4 Environmental Robustness

As stated in the High-Level Design Report under design goal G4, environmental durability (dust, moisture, temperature variation) was a design intention rather than a validated property. No comprehensive ingress-protection or temperature-cycling testing was performed on the prototype, and the report does not claim such certification. The wearable form factor exposes wiring and modules that would, in a production unit, need encapsulation. This limitation is acknowledged explicitly and is enumerated under future work in §10.2.

5. New Tools and Technologies Used

The AKKE project exposed the team to several technologies that were outside the standard undergraduate Computer Engineering & Electrical and Electronics Engineering curriculum at TED University. This section summarizes the principal ones.

5.1 Edge Impulse (TinyML Platform)

Edge Impulse is a cloud-based development platform for embedded machine-learning pipelines. Compared to writing the entire training pipeline by hand in TensorFlow, Edge Impulse offers three concrete advantages that were exploited in this project: (i) it provides a graphical interface for managing time-windowed datasets, including drag-and-drop ingestion of per-recording CSV files and automatic train/test splitting; (ii) it generates a fully self-contained Arduino library, complete with int8-quantised model weights and a portable C++ inference runtime that compiles cleanly under the Arduino-ESP32 core; and (iii) it emits the integer constants (window size, sample frequency, number of features, number of labels) as compile-time macros, which lets the firmware refer to the model's hyper-parameters without ever hard-coding them. The deployed library version is `ei-ilter-akke_1.3.3-arduino-1.0.3-impulse-#1.zip`.

5.2 ESP-NOW Protocol

ESP-NOW is a connectionless peer-to-peer Wi-Fi protocol developed by Espressif Systems. It reuses the radio hardware of the ESP32 family but bypasses the standard Wi-Fi association procedure, eliminating the join, DHCP, and routing latency that ordinary Wi-Fi imposes. Packets up to 250 bytes can be sent to a peer addressed by its MAC address, with per-packet delivery acknowledgement available through a callback. For AKKE, ESP-NOW reduces the wireless command latency to single-digit milliseconds and removes any dependence on an access point. The team learned to handle the dual-callback model (OnDataSent on the glove side, OnDataRecv on the receiver side) and to navigate the ESP-IDF v5 API change that altered the signature of OnDataRecv; the receiver firmware uses a preprocessor branch on `ESP_IDF_VERSION_MAJOR` to remain compatible with both API versions.

5.3 Kalman Filtering for IMU Fusion

The deployed firmware uses a one-dimensional Kalman filter for each of the pitch and roll channels, fusing accelerometer-derived angle measurements with gyroscope rate measurements. The implementation is the open-source TKJ Electronics Kalman library bundled in the repository under ESP32-Libraries/Kalman.zip. Working with the Kalman filter required the team to compute the time delta Δt in seconds between successive IMU samples, to convert the gyroscope raw values to physical rates in degrees per second (division by 131 for the ± 250 °/s setting), and to seed the filter's internal angle estimate from the first valid accelerometer reading. The filtered angles are markedly less noisy than the raw accelerometer-derived angles, which improves the separability of gestures that differ chiefly in wrist orientation.

5.4 DFPlayer Mini MP3 Module

The DFPlayer Mini is a self-contained MP3 decoder that accepts UART control commands and plays files from a microSD card. The team learned the protocol-level details that matter for AKKE: the 9600 baud asynchronous serial interface (used here on HardwareSerial 1 with TX on GPIO 17 and RX on GPIO 16); the difference between `play(n)` and `playMp3Folder(n)` (only the latter addresses files in `/mp3/` by their numeric prefix, which is what AKKE relies on); and the module's tendency to enter an error state if the SD card is missing, which is handled explicitly in the receiver setup.

5.5 Other Tooling

Additional tools and technologies adopted during the course include:

- Arduino IDE 2.x and the Arduino-ESP32 core (v3.x) for firmware development.
- `pyserial` for the host-side UART data-collection script (`data_collector.py`).
- The Edge Impulse CLI (`edge-impulse-uploader`, distributed via `npm`) for batch ingestion of per-class CSV files into the Edge Impulse project.
- MPU-6050 register-level programming via I2C using the raw `Wire` library, rather than a high-level wrapper, to retain explicit control over sample-rate divider, full-scale settings, and digital low-pass filter.
- 3D printing of the receiver enclosure.

5.6 Power Consumption and Runtime Characteristics

The AKKE prototype was designed as a low-power embedded wearable system capable of operating fully offline without external infrastructure. Although detailed laboratory-grade power instrumentation was outside the scope of the project, approximate current-consumption measurements and runtime estimations were obtained using USB current meters and component datasheet references.

The glove and receiver subsystems are each powered by independent 9 V alkaline batteries through buck converters that provide regulated 5 V output to the ESP32 development boards. The ESP32 onboard regulators then generate the required 3.3 V logic rail.

Table X.X summarizes the approximate operating current observed in the deployed prototype.

Subsystem / Mode	Approximate Current Draw
Glove – Idle / READY	80–100 mA
Glove – Active inference window	140–180 mA
Glove – ESP-NOW transmission peak	220–260 mA
Receiver – Idle	90–110 mA
Receiver – Audio playback (medium volume)	180–250 mA
Receiver – Maximum audio peak	300–350 mA

The dominant transient current peaks occur during Wi-Fi-radio transmission bursts on the ESP32 and during DFPlayer-driven speaker playback at high volume. The TinyML inference step itself contributes comparatively little additional power consumption relative to the ESP32 base operating current.

Using a typical alkaline 9 V battery capacity of approximately 500–600 mAh under low-to-moderate load conditions, the practical operating lifetime of the prototype is estimated as follows:

Device	Estimated Runtime
Glove unit	~2–4 hours intermittent tactical use
Receiver unit	~1.5–3 hours depending on playback frequency and volume

These values are approximate because alkaline battery discharge behavior varies significantly with pulse current, temperature, and converter efficiency. The project therefore does not claim formally validated battery endurance figures.

5.7 Memory Footprint and Embedded Resource Usage

A central design constraint of AKKE was the ability to execute all sensing, preprocessing, and machine-learning inference directly on the ESP32-WROOM-32 without external computation resources.

The deployed Edge Impulse model was exported as an int8-quantized TensorFlow Lite Micro inference library and integrated into the firmware through the generated Arduino package. Quantization substantially reduced both flash usage and RAM consumption compared to the original float32 reference model.

The approximate embedded resource usage of the deployed firmware is summarized below.

Resource	Approximate Usage
Flash usage (total firmware)	~1.3–1.6 MB
Edge Impulse model size	~200–350 KB
Tensor arena / inference buffer	~80–120 KB RAM
Runtime SRAM usage	~220–300 KB
ESP32 available SRAM	520 KB

The largest memory consumers are:

1. the TensorFlow Lite Micro tensor arena,
2. the 100×13 floating-point feature buffer,
3. Edge Impulse DSP preprocessing buffers,
4. Wi-Fi / ESP-NOW runtime allocations.

The int8 quantization step was therefore critical for deployment feasibility. The float32 model variant achieved slightly higher accuracy during experimentation but consumed significantly more memory and inference time, making it less suitable for reliable deployment on the ESP32-WROOM-32 platform.

The deployed model comfortably fits within the ESP32 memory budget while leaving sufficient headroom for:

- ESP-NOW communication,
- Kalman filtering,
- sensor acquisition,
- LED/UI control,
- serial debugging.

This validates the suitability of TinyML workflows for low-cost embedded wearable systems without requiring higher-end microcontrollers or external AI accelerators.

6. Test Results

6.1 Test Approach

Testing was carried out in accordance with the Test Plan v2 deliverable submitted earlier in the spring semester. The plan categorized tests into four groups: (i) unit tests at the firmware-driver level (sensors, UART, I2C, ADC), (ii) component tests at the subsystem level (calibration, state machine, ESP-NOW link, DFPlayer playback, LCD UI), (iii) machine-learning model tests evaluated against the held-out test set, and (iv) end-to-end integration tests that exercise the entire pipeline from gesture to audio output. Tests were performed manually for the firmware and hardware categories, with results recorded on a structured spreadsheet, and automatically through Edge Impulse’s model-evaluation tooling for the ML category.

6.2 Functional Test Cases

Table 6.1 lists the principal functional test cases executed on the assembled prototype.

Test ID	Description	Expected Result	Outcome
T-01	Boot sequence on glove: ESP32 initializes serial at 115200 baud, I2C at 400 kHz, ADC at 12-bit / 11 dB.	Serial banner printed; no I2C error.	Pass
T-02	Flex sensor read on each of the five channels while bending the finger.	Raw ADC values transition smoothly between extended and flexed positions.	Pass
T-03	MPU-6050 detection at I2C address 0x68.	All 14 bytes from register 0x3B are returned; no I2C timeout.	Pass
T-04	Three-stage calibration with correct LED indications (yellow → red → magenta).	LED transitions match spec; cal_open, cal_close, and acc/gyro offsets populated.	Pass
T-05	Edge Impulse classifier constants reported correctly in the boot banner.	EI_CLASSIFIER_FREQUENCY = 50, RAW_SAMPLE_COUNT = 100, LABEL_COUNT = 10.	Pass

T-06	Inference on a clearly held “Stop” gesture.	Top-1 label is Hareket_1 with confidence ≥ 0.75 ; command 1 transmitted.	Pass
T-07	Inference on a deliberately ambiguous mid-motion pose.	Confidence below 0.75; red LED blink; no ESP-NOW transmission.	Pass
T-08	ESP-NOW send callback reports BASARILI status on every transmission.	OnDataSent prints success; receiver acknowledges packet.	Pass
T-09	Receiver boots, mounts SD via DFPlayer, reaches “command-waiting” state.	LCD shows “Sistem Baslatiliyor” then “Bekleniyor...”.	Pass
T-10	Receiver plays the correct MP3 file for each of the ten commands.	Each command 1–10 plays /mp3/0001.mp3 ... /mp3/0010.mp3.	Pass
T-11	Receiver LCD updates command text after every received command.	Line 1 shows the latest command in human-readable form.	Pass
T-12	Volume potentiometer adjusts DFPlayer volume in 0–30 range and updates LCD.	LCD volume value tracks the pot; audio level changes audibly.	Pass
T-13	End-to-end: ten consecutive successful gestures after calibration.	Each gesture produces the expected audio within ≤ 1 s after gesture completion.	Pass
T-14	Range test: receiver at 10 m with line-of-sight to the glove.	All transmitted packets received and played.	Pass
T-15	Range test: receiver at ≈ 20 m with line-of-sight.	Packets received without retransmission needed.	Pass

T-16	Range test: receiver at 10 m through one interior wall.	Packets received; occasional packet loss in worst orientation, recovered on retry.	Partial
T-17	Invalid payload (out-of-range command ID) injected at receiver.	Receiver logs a warning and does not play any audio.	Pass
T-18	SD card removed before boot.	Receiver enters DFPlayer error state, LCD shows “DFPlayer Hata / Kontrol et”.	Pass
T-19	Battery monitoring on the glove through GPIO 39 voltage-divider input.	ADC reading is converted into a plausible battery voltage/percentage; red LED warning is activated under low-battery condition.	Pass

Table 6.1. Functional and integration test results.

6.3 Machine-Learning Model Performance

The deployed int8-quantised Edge Impulse model was evaluated on a held-out validation set of 813 samples. The Quantized (int8) variant — the same model version exported to the Arduino inference firmware — achieved an overall accuracy of 94.34%, with weighted-average precision, recall, and F1-score values of 0.944, 0.943, and 0.944 respectively. The one-vs-rest ROC-AUC score was 0.995.

The float32 reference variant achieved 95.57% accuracy on the same validation set. Therefore, int8 quantization introduced an accuracy reduction of approximately 1.23 percentage points. This decrease was considered acceptable because the int8 model is significantly more suitable for deployment on the ESP32-WROOM-32 in terms of memory footprint and inference cost.

All ten gesture classes exceeded the original 75% recognition target. The strongest classes were Stick Together, Stop, Move Right, Come, and Move Left, while the weakest classes were Fall Back, Slow Down, and Take Cover. Fall Back obtained the lowest F1-score at 0.841. The confusion matrix shows that most errors occur among the downward-trajectory gesture group: Slow Down, Fall Back, and Take Cover. This is consistent with the physical similarity of these gestures and with the partial overlap visible in the Edge Impulse Feature Explorer.

When the confidence threshold is applied, Edge Impulse marks some low-confidence predictions as uncertain. In the Edge Impulse UI, this confidence-gated evaluation reports 91.37% accuracy. This value should be interpreted separately from the raw classifier accuracy of 94.34%: the former includes confidence-based rejection, while the latter reports the standard int8 classifier performance over the 813-sample validation set.

Gesture (Internal Label)	Precision	Recall	F1-score	Support
Stop (Hareket_1)	0.988	0.988	0.988	81
Listen (Hareket_2)	0.950	0.927	0.938	123
Go go (Hareket_3)	1.000	0.960	0.980	75
Come (Hareket_4)	1.000	0.984	0.992	61
Stick together (Hareket_5)	0.935	0.960	0.947	75
Slow down (Hareket_6)	0.882	0.893	0.888	84
Fall back (Hareket_7)	0.846	0.835	0.841	79
Take cover (Hareket_8)	0.881	0.952	0.915	62
Move right (Hareket_9)	0.990	0.971	0.980	103
Move left (Hareket_10)	0.958	0.986	0.972	70
Macro average	0.943	0.945	0.944	813
Weighted average	0.944	0.943	0.944	813

Table 6.2. Per-class performance of the deployed int8 model.

Every gesture clears the 75% recognition-accuracy target stipulated in REQ-F-006 of the Project Specifications Report. Fall Back (F1 = 0.841), Slow Down (F1 = 0.888), and Take Cover (F1 = 0.915) are the weakest classes; the confusion matrix in Appendix B shows that the principal source of error is mutual confusion within this downward-trajectory triad, three gestures that share a similar terminal forearm posture. Listen is also occasionally misclassified as Go Go (4 cases) and as Slow Down (2 cases), both of which end with the hand in a partially open pose. At the operational level, Edge Impulse's confidence threshold flags a portion of these low-confidence predictions as "Uncertain" rather than committing to a prediction; the firmware's confidence gate of §2.6 then converts these into "no command transmitted" outcomes, which is the safety-relevant behavior we want in a tactical context. These confusion patterns are consistent with the t-SNE visualization of the feature space in Edge Impulse's Feature Explorer, in which the corresponding clusters partially overlap.

6.4 Performance Tests

Latency was measured by instrumenting the glove firmware to print `millis()` at the moment of the button press and at the moment ESP-NOW returned its delivery acknowledgement. The dominant term is the 2-second sensor capture window, which is intentional: it is the design parameter that defines the system's observation interval. After the window completes, the Edge Impulse inference step on the ESP32-WROOM-32 takes tens of milliseconds, and the ESP-NOW transmission and DFPlayer trigger together add a handful of milliseconds. The total end-to-end latency from the start of the capture window to the first audible byte of the played MP3 is therefore approximately 2.05–2.15 s in practice; from the moment the user completes the gesture (i.e. roughly the center or end of the capture window, depending on timing) to playback is comfortably under one second, satisfying the < 1 s responsiveness goal stated in HLD design goal G2.

6.5 Test Statistics and Assessment

Of the nineteen functional test cases enumerated in Table 6.1, eighteen are recorded as Pass and one (T-16, range through one interior wall) as Partial. The partial outcome reflects intermittent packet loss in the worst-case orientation behind a load-bearing concrete-and-rebar wall; in the open-field range test (T-15) at twenty meters, no losses were observed. The two noteworthy bugs encountered and resolved during testing were:

- An early version of the inference sketch used hard-coded literals (`delay(20)`, a fixed for-loop bound of 100) instead of the Edge Impulse compile-time macros. When the team experimented with a re-trained model at a different sampling frequency, the firmware silently ran at 50 Hz while the model expected the new value, and inference confidence dropped to roughly the uniform-random level across all classes. Replacing the literals with `EI_CLASSIFIER_FREQUENCY`, `EI_CLASSIFIER_RAW_SAMPLES_PER_FRAME`, and `EI_CLASSIFIER_RAW_SAMPLE_COUNT` eliminated the problem and is the form that ships in the final firmware.
- The synthetic-data experiment described in §3.3 produced a measurable regression in validation accuracy and was rolled back. The lesson is captured in the project's engineering notes and informed the final “real data only” policy.

Potential enhancements that emerged from testing are listed in §10.2 as future work.

7. Impact of Engineering Solutions

7.1 Global Context

Silent and reliable command-and-control communication is a problem of global relevance. Military, law-enforcement, and emergency-response teams in every country face situation in which speech is unsafe, impractical, or impossible: high-noise industrial accidents, hostage and breaching operations, fire response in burning buildings, and underwater or vacuum work. Traditional alternatives — visual hand signals, hard-wired throat microphones, or radio voice traffic — each have well-known failure modes. AKKE contributes a small but concrete example of how on-device machine learning and wearable sensors can make silent communication faster, more disciplined, and less dependent on direct line-of-sight. The hardware and software stack employed (ESP32, MPU-6050, flex sensors, Edge Impulse, ESP-NOW) is intentionally built on commodity components and an open development tool-chain, which makes the design reproducible by similar undergraduate and industry teams worldwide.

7.2 Economic Context

The total bill of materials for the AKKE prototype is well under one hundred US dollars per glove and per receiver, driven mainly by the choice of the ESP32-WROOM-32 (single-digit dollars) over more expensive microcontrollers, and by the use of consumer-grade flex sensors and IMU breakouts. Compared to commercial tactical communication systems, which can cost thousands of dollars per operator, the cost reduction is two to three orders of magnitude. While AKKE is not a finished industrial product, it demonstrates that a useful subset of the functionality of such systems can be implemented at a price point compatible with deployment by smaller teams — reservists, private security, industrial workers — who would otherwise have no access to silent command-and-control hardware at all. The use of an off-line, on-device classifier also means that the system has no per-month cloud-service cost.

7.3 Environmental Context

AKKE operates fully off-line. The system is air-gapped: it does not require Wi-Fi association, cellular service, or Internet connectivity. From the perspective of network and energy infrastructure, this means zero contribution to data-center electricity use during operation. Power is supplied by a 9 V alkaline (zinc–manganese dioxide) primary battery on each subsystem through a buck converter with a regulated 5 V USB output. In the prototype, this 5 V output is connected to the ESP32 development board through its micro-USB port, and the board’s onboard regulator provides the 3.3 V logic rail required by the microcontroller and low-voltage peripherals. The prototype is powered by 9 V alkaline (single-use, primary) cells, which were selected for their immediate availability, simple handling without charging electronics, and robust performance across temperature. The team explicitly acknowledges that primary alkaline chemistry incurs higher per-operation e-waste than a rechargeable lithium chemistry; this trade-off is documented as a productization concern in §10.2 rather than a property of the present demonstration unit. On the other hand, in a productized version the team would specify a rechargeable lithium chemistry (Li-ion or LiFePO₄) with appropriate charging electronics and a clearly labelled, replaceable cell to reduce per-operation e-waste relative to the disposable alkaline cells used in the prototype. The wearable form factor itself contains no exotic materials beyond what is standard in consumer electronics.

7.4 Societal Context

AKKE addresses a societal need most acutely felt by individuals whose work requires them to operate in conditions where speech is unsafe, including the hearing-impaired community when extended to a gesture-to-speech application. Although the present project is framed around tactical use, the underlying capability — reliable, low-latency translation of a small vocabulary of hand gestures into audible commands — generalizes to sign-language augmentation, accessibility tools, and silent operation of industrial machinery in noisy environments. The system’s confidence gate, which refuses to broadcast an uncertain command rather than risk a wrong one, is also a deliberate societal-safety choice: in a domain where, mistaken commands could harm people, “no command” is always preferable to “wrong command,” and the architecture is written accordingly.

7.5 Ethical Considerations

Because AKKE is intended for tactical use, the team paid explicit attention to misuse risk and consent. The system is designed for use by authorized, trained personnel within defined operational roles; the command set is deliberately limited and is bound to a specific receiver via a hard-coded MAC address. The system never operates on personally identifying biometric data: the only signals leaving the glove are flex-sensor and IMU readings during data collection, which never leave the user's host computer, and a one-byte command identifier during operation, which contains no user information.

8. Contemporary Issues Related to the Project

8.1 The Rise of TinyML and Edge AI

AKKE is a small example of a much wider trend in contemporary computing: the migration of machine-learning inference from cloud data centers onto microcontrollers physically embedded in everyday objects. “TinyML” — the practice of running quantized neural networks on devices with kilobytes of RAM and tens of milliwatts of power — has matured rapidly in the last several years, driven by the maturation of TensorFlow Lite Micro, the emergence of platforms such as Edge Impulse, and the explosion in low-cost ARM Cortex-M and Tensilica Xtensa microcontrollers. The technical appeal is obvious: local inference removes network round-trips, eliminates the need for an Internet connection, and gives strong locality guarantees over user data. The contemporary policy debate, however, is less settled. As edge AI proliferates, regulators and standards bodies are now grappling with questions of explainability, accountability, and software-supply-chain integrity for models that run inside devices the end-user never inspects directly. AKKE’s confidence gate and the explicit serial print-out of every classification result are small steps in the direction of explainable, auditable on-device inference.

8.2 Wearable Computing for Tactical and Industrial Use

Wearable computing for first responders, military personnel, and industrial workers is one of the fastest-growing segments of the wearable market. Programmes such as the U.S. Army’s Integrated Visual Augmentation System (IVAS) and various European “soldier-modernization” initiatives all include some form of hand-mounted input device for silent command. The contemporary research literature on smart gloves — e.g. the 2024 IEEE Transactions on Human-Machine Systems review by Belcamino et al. and the same year’s open-access work by Filipowska et al. in *Sensors* — consistently identifies three open problems: sensor variability, user-to-user generalization, and robustness against environmental disturbance. AKKE encountered all three: flex-sensor variability between channels degraded early models; the per-session calibration routine is the project’s response to user-to-user variation; and environmental robustness remains an explicit limitation of the present prototype.

8.3 Silent and Low-Latency Communication Channels

The choice of ESP-NOW over Wi-Fi or Bluetooth was driven by latency and overhead, but it also intersects with the contemporary discussion of ad-hoc, low-overhead, short-range communication. Standards such as the recently published IEEE 802.11bb (“Li-Fi” optical wireless) and the continuing development of 802.11ah (“HaLow”) reflect the demand for low-power, low-overhead links that bypass the conventional access-point model. ESP-NOW is not standardised and is vendor-specific, but it exemplifies the same idea — a connectionless peer-to-peer model that trades the generality of TCP/IP networking for predictable, low-latency control communication. The cost is reduced interoperability with non-Espressif hardware; the benefit is the operational simplicity that makes AKKE feasible as a senior project.

8.4 Privacy, Bias, and Inclusivity in Gesture Recognition

Any machine-learning system trained on data from a small number of users runs the risk of failing on users whose physiology or motion patterns lie outside the training distribution. The AKKE dataset was collected primarily from the project team, which biases the model toward right-handed, able-bodied, average-sized adult hands. A productised version of the system would need to broaden the training population substantially and would also need to address left-handed operation (which currently requires either re-training or a mirrored flex-sensor layout). These issues are part of the broader contemporary conversation about fairness and inclusivity in machine-learning systems, and they are recognized explicitly in the IEEE Ethically Aligned Design framework.

9. Background Research and Resource Usage

9.1 Similar Smart-Glove Designs

The team carried out a systematic literature review of similar smart-glove designs at the start of the project. The two most influential references were the 2024 IEEE Transactions on Human-Machine Systems systematic review by Belcamino, Carfi, and Mastrogiovanni [4] and the 2024 Sensors article by Filipowska et al. on machine-learning-based gesture-recognition gloves [8]. The Belcamino review surveyed sensor modalities (flex, capacitive, inertial, electromyographic) and their typical accuracy/latency trade-offs, and informed AKKE's choice to combine flex with IMU. The Filipowska paper described a 3D-printed glove with five flex sensors and per-user calibration, an architecture broadly similar to AKKE's; its description of normalisation pitfalls informed the team's decision to normalize flex on-device before serialization, and not in post-processing. Additional context was drawn from the 2024 Scientific Reports article by Leite et al. on simulation-driven design of smart gloves [10], and from Ong's 2017 project report on data-glove gesture recognition with flex and IMU [11], the latter of which served as a structural template for the data-collection protocol.

9.2 Component Datasheets and Reference Materials

Component-level engineering decisions were anchored on the manufacturers' datasheets:

- Espressif Systems, ESP32-WROOM-32 Datasheet v3.3 [5]: pin map, ADC characteristics, deep-sleep current, and reset behavior.
- InvenSense, MPU-6000 / MPU-6050 Product Specification Rev. 3.4 [6]: register map (in particular 0x19, 0x1B, 0x1C, 0x6B), sample-rate divider arithmetic, and self-test procedure.
- Espressif Systems, ESP-NOW User Guide, ESP-IDF Programming Guide [7]: peer registration, callback signatures, and the v5 API change for OnDataRecv.
- DFRobot, DFPlayer Mini MP3 Player for Arduino (SKU: DFR0299) [9]: UART command set, file-numbering conventions, and the difference between play(n) and playMp3Folder(n).

9.3 Engineering Principles and Software Methodology

The structural backbone of the design documents follows Bruegge and Dutoit’s “Object-Oriented Software Engineering: Using UML, Patterns, and Java” (2nd ed.) [3], which is the recommended reference in the CMPE 491 and 492 syllabi. Use-case diagrams, sequence diagrams, deployment diagrams, and the subsystem-decomposition technique used throughout this report and its predecessors are all drawn from that source. The layered-architecture style used to organize the firmware — hardware abstraction layer, data-processing layer, application layer — is also a textbook pattern. Fowler’s “UML Distilled” [12] was used as a quick reference for diagram syntax.

9.4 Online Resources Consulted

In addition to the primary references above, day-to-day engineering questions were answered using a small set of online resources: the Edge Impulse documentation portal (for SDK API and deployment guides); the Arduino-ESP32 GitHub repository (especially the WiFi and esp_now examples); the Adafruit and Sparkfun learn pages (for I2C wiring of the MPU-6050 and LCD modules); and the Stack Overflow community for diagnosis of compiler errors during the transition between Arduino-ESP32 v2.x and v3.x. The project website hosts the up-to-date report versions and is publicly accessible at <https://ilter-akke.github.io/website/>. The complete implementation repository — including firmware sketches, the consolidated dataset, the Edge Impulse model export, and all supporting Python scripts — is publicly available at <https://github.com/ILTER-AKKE/AKKE-Smart-Command-Control-Glove>.

10. Discussion and Future Work

10.1 Discussion

AKKE achieves all of its stated functional requirements: it recognizes ten distinct hand gestures with an aggregate int8 test accuracy of 94.34% on a held-out test set of 813 samples, transmits the recognized command over a low-latency wireless channel, and reproduces the command as an audible message on a receiver positioned at least ten meters away, all without dependence on external network infrastructure. Three observations stand out from the engineering experience.

First, the discipline of forcing the firmware to consume the Edge Impulse compile-time constants rather than hard-coded literals turned out to be one of the most valuable architectural decisions of the project. It eliminated a class of silent bugs in which a re-trained model and an out-of-sync firmware would coexist on the same device and silently produce nonsense predictions. The serial banner that prints the constants at boot has become the team's first sanity check whenever an unexpected accuracy result is observed.

Second, the synthetic-data experience was a productive failure. It taught the team that the instinct to “amplify” a real dataset with statistically plausible variations is dangerous when the underlying sensors have non-Gaussian, hardware-specific noise. The right tool for the AKKE situation was — and remains — collecting more real samples under varied conditions, not synthesizing them. This is now a documented project policy.

Third, the per-session calibration routine, although it adds approximately eight seconds to the wake-up time of the glove, paid for itself many times over in inference robustness. Without calibration, the flex-sensor distributions seen at inference time diverge measurably from those in the training set, and accuracy degrades. With calibration, the glove can be donned by different team members in succession without any retraining.

10.2 Future Work

Five concrete enhancements are identified as future work, in approximate order of priority:

- Extend the gesture vocabulary to the full fifteen-gesture catalogue listed in README.md (adding Check the area, Spread out, Get down, Surroundings are clean, and Operation is over). This is primarily a data-collection and re-training task.
- Disambiguate the Slow Down, Fall Back, and Take Cover cluster identified in the confusion matrix, either by collecting more data for those classes or by introducing an auxiliary feature (e.g. forearm pitch over the entire window rather than only at the endpoints).
- Productize the wearable form factor with encapsulated wiring, environmental sealing, a low-profile PCB to replace the breadboard-style wiring of the current prototype, and proper ingress-protection (IP) testing.
- Add over-the-air model update capability. Today, a re-trained model requires the team to rebuild and flash the Arduino firmware. ESP-IDF provides standard OTA update primitives that would allow the model header file to be updated over the same ESP-NOW channel or via a separate dedicated upload mode.
- Strengthen the wireless security model by adding payload-level authentication (e.g. AES-128 CCM) on top of ESP-NOW. The current implementation has no authentication beyond the implicit MAC-address peer registration, which would be inadequate for a fielded operational unit.

11. Conclusion

The AKKE Smart Command and Control Glove is a working, end-to-end demonstration of how an undergraduate team can integrate wearable sensing, on-device machine learning, low-latency wireless communication, and embedded audio playback into a single coherent system for a real operational use case. The project began with an ambitious specification, evolved through a high-level design that was subsequently revised on the basis of practical experience, and ended with a deployed prototype whose behavior is fully documented in the firmware and dataset committed to the project's GitHub repository.

Measured against its own requirements, AKKE meets or exceeds every specified functional and non-functional criterion: it recognizes ten distinct gestures rather than the originally specified five; it achieves 94.34% int8 accuracy against a 75–80% target, with every individual gesture clearing the per-class threshold, and with the lowest F1-score being 0.841 for Fall Back. It transmits over at least ten meters of line-of-sight, with intermittent through-wall performance; and its post-gesture response time from gesture-window completion to audio playback is well under one second. The confidence-gated inference loop ensures that the system fails silently rather than incorrectly when faced with ambiguous input, which is the safety-critical property required for a tactical communication device.

Beyond the technical outcomes, the project taught the team how to reconcile evolving design documents with operational reality, how to architect firmware so that machine-learning hyper-parameters and code stay synchronized, and how to respect the difference between real and synthetic sensor data. These are durable engineering lessons that will outlast the specific implementation. Both the project website (<https://ilter-akke.github.io/website/>) and the implementation repository (<https://github.com/ILTER-AKKE/AKKE-Smart-Command-Control-Glove>), including the firmware, dataset, and trained model files, are publicly accessible.

The team hopes that the documented architecture and lessons learned will serve as a useful reference for other student groups working on TinyML-driven wearable systems.

12. Glossary / Acronyms

This section defines the main technical acronyms and abbreviations used throughout the AKKE Final Report.

Acronym / Term	Meaning
ADC	Analog-to-Digital Converter. A hardware unit that converts analog sensor voltages into digital values readable by the microcontroller. In AKKE, the flex sensors and battery voltage dividers are read through ESP32 ADC pins.
AES-128 CCM	Advanced Encryption Standard – 128-bit Counter with CBC-MAC Mode. A cryptographic mode that provides both encryption and authentication. It is mentioned as a possible future enhancement for securing ESP-NOW payloads.
AKKE	Smart Command and Control Glove project name. It refers to the wearable gesture-recognition system developed by the team.
API	Application Programming Interface. A defined interface that allows software components to communicate. In the report, it appears mainly in relation to ESP-IDF / Arduino libraries and callback functions.
CNN / Conv1D	Convolutional Neural Network / One-Dimensional Convolution. A neural-network architecture suitable for time-series classification. AKKE uses a compact Conv1D-based model for gesture recognition.
CSV	Comma-Separated Values. A tabular text-file format used for storing the collected gesture dataset.

Acronym / Term	Meaning
DFPlayer Mini	A small MP3 playback module controlled over UART. In AKKE, it plays the pre-recorded audio command files stored on a microSD card.
DSP	Digital Signal Processing. Processing applied to digital sensor signals before classification. In AKKE, this includes feature construction, normalization, and time-window processing.
EI	Edge Impulse. The TinyML platform used to train, evaluate, quantize, and export the gesture-recognition model.
ESP-IDF	Espressif IoT Development Framework. The official development framework for ESP32 devices. The report references ESP-IDF especially in relation to ESP-NOW API changes and OTA support.
ESP-NOW	A connectionless peer-to-peer wireless communication protocol developed by Espressif. AKKE uses ESP-NOW to transmit a one-byte command identifier from the glove to the receiver without Wi-Fi association or Internet access.
ESP32-WROOM-32	The ESP32 module used in both the glove and receiver units. It provides the microcontroller, Wi-Fi radio, ADC, UART, I2C, and enough processing capability for on-device inference.
F1-score	A machine-learning evaluation metric that combines precision and recall. It is used in the report to evaluate per-class gesture-recognition performance.
GPIO	General-Purpose Input/Output. Programmable microcontroller pins used to read sensors, control LEDs, communicate with modules, and monitor battery voltage.
HLD	High-Level Design. The earlier design document that described the initial system architecture before implementation revisions.

Acronym / Term	Meaning
I2C	Inter-Integrated Circuit. A two-wire serial communication protocol using SDA and SCL lines. In AKKE, it is used for the MPU-6050 IMU and the 16×2 LCD screen.
IMU	Inertial Measurement Unit. A sensor module that measures acceleration and angular velocity. AKKE uses the MPU-6050 IMU to capture hand orientation and motion.
IP	Ingress Protection. A rating system describing resistance to dust and water. The report mentions IP testing as future work for environmental robustness.
ISM Band	Industrial, Scientific, and Medical Band. A license-free radio-frequency band. ESP-NOW operates in the 2.4 GHz ISM band.
LCD	Liquid Crystal Display. The receiver uses a 16×2 LCD to show the latest command and volume level.
LLD	Low-Level Design. The design document that refined the HLD and moved closer to the final implemented hardware and firmware.
LSB	Least Significant Bit. In the report, it is used as the raw measurement unit for accelerometer and gyroscope readings from the MPU-6050.
MAC Address	Media Access Control Address. A unique hardware address used to identify a wireless device. AKKE uses the receiver's MAC address for ESP-NOW peer registration.
ML	Machine Learning. The field of using data-driven models to make predictions. AKKE uses ML to classify hand gestures from sensor data.
MP3	A compressed digital audio format. AKKE stores spoken command audio files as numbered MP3 files on the receiver's microSD card.

Acronym / Term	Meaning
MPU-6050	The IMU module used in AKKE. It contains a three-axis accelerometer and a three-axis gyroscope.
OTA	Over-the-Air Update. A method of updating device firmware wirelessly. OTA model or firmware updates are listed as future work.
PCB	Printed Circuit Board. A custom board that could replace prototype wiring in a productized version of AKKE.
RAM / SRAM	Random Access Memory / Static Random Access Memory. Memory used by the microcontroller during runtime. The ESP32-WROOM-32 provides sufficient SRAM for sensing, filtering, and inference.
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve. A machine-learning metric used to evaluate classification separability. The report gives a one-vs-rest ROC-AUC score for the model.
SD / microSD	Secure Digital / micro Secure Digital. A removable storage medium. The receiver stores the MP3 command files on a microSD card.
Softmax	A neural-network output function that converts model scores into class probabilities. AKKE’s model uses softmax over ten gesture classes.
TCP/IP	Transmission Control Protocol / Internet Protocol. Standard network communication protocols. AKKE does not rely on TCP/IP during operation because ESP-NOW is used instead.
TinyML	Tiny Machine Learning. The practice of running machine-learning models on low-power embedded devices. AKKE runs a quantized gesture-recognition model directly on the ESP32.

Acronym / Term	Meaning
UART	Universal Asynchronous Receiver/Transmitter. A serial communication protocol. AKKE uses UART to control the DFPlayer Mini module and for USB serial communication during data collection/debugging.
USB	Universal Serial Bus. Used for programming, debugging, and powering the ESP32 development boards through their micro-USB connectors.
Wi-Fi	A wireless networking technology based on IEEE 802.11 standards. AKKE uses the ESP32 Wi-Fi radio hardware for ESP-NOW but does not connect to a Wi-Fi network

13. References

- [1] AKKE Team, "Project Specifications Report," CMPE 491, TED University, Fall 2025.
- [2] AKKE Team, "Analysis Report and High-Level Design Report," CMPE 491, TED University, Fall 2025.
- [3] B. Bruegge and A. H. Dutoit, Object-Oriented Software Engineering: Using UML, Patterns, and Java, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2003.
- [4] A. Belcamino, M. Carfi, and F. Mastrogiovanni, "Wearable sensing for hand-gesture recognition: a systematic review," IEEE Transactions on Human-Machine Systems, 2024.
- [5] Espressif Systems, ESP32-WROOM-32 Datasheet, Version 3.3, 2023.
- [6] InvenSense, MPU-6000 and MPU-6050 Product Specification, Revision 3.4, 2013.
- [7] Espressif Systems, ESP-NOW User Guide, ESP-IDF Programming Guide, 2024. [Online]. Available: <https://docs.espressif.com>
- [8] J. Filipowska et al., "Machine-learning-based gesture recognition glove with flex sensors and IMU," Sensors, vol. 24, 2024.
- [9] DFRobot, DFPlayer Mini MP3 Player for Arduino (SKU: DFR0299) Product Wiki, 2024. [Online]. Available: <https://wiki.dfrobot.com>
- [10] L. Leite et al., "Simulation-driven design of a smart sensing glove," Scientific Reports, vol. 14, 2024.
- [11] J. Ong, "Data glove for sign-language gesture recognition with flex and IMU sensors," B.Sc. project report, 2017.
- [12] M. Fowler, UML Distilled, 3rd ed. Boston, MA: Addison-Wesley, 2003.
- [13] IEEE, IEEE Code of Ethics. [Online]. Available: <https://www.ieee.org/about/corporate/governance/p7-8.html>
- [14] ACM, ACM Code of Ethics and Professional Conduct, 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [15] Edge Impulse, Edge Impulse Documentation. [Online]. Available: <https://docs.edgeimpulse.com>
- [16] TKJ Electronics, Kalman Filter Library for Arduino. [Online]. Available: <https://github.com/TKJElectronics/KalmanFilter>
- [17] AKKE Team, "AKKE Implementation Repository," GitHub, 2026. [Online]. Available: <https://github.com/ILTER-AKKE/AKKE-Smart-Command-Control-Glove>

Appendix A: Implementation Repository Structure

The implementation repository is publicly available at <https://github.com/ILTER-AKKE/AKKE-Smart-Command-Control-Glove>. Its internal structure is summarized below to document the organization of the firmware, dataset, model files, and supporting scripts used during development.

Path	Description
README.md	Original Turkish data-collection plan and gesture catalogue.
inference/inference.ino	Final glove firmware (Arduino sketch). Reads sensors, runs Edge Impulse inference, applies confidence gate, transmits ESP-NOW packet.
inference/model.h	Generated TFLite model header for the reference local pipeline.
reciever.ino	Final receiver firmware. Receives ESP-NOW packet, plays MP3 via DFPlayer, updates LCD.
data_collection/data_collection.ino	Data-collection firmware. Same pin map and feature definition as inference but writes DATA lines to UART for the host script.
data_collection/data_collector.py	Python host script that reads UART, auto-labels by record_id, writes dataset.csv.
data_collection/count_labels.py	Utility script that reports per-class sample counts.
data_collection/convert_dataset.py	Utility that converts the consolidated dataset into the per-recording CSV layout expected by Edge Impulse.
data_collection/dataset.csv	Consolidated dataset (725,550 rows, 4,837 recordings, 10 classes).
data_collection/edge_impulse_data/	Per-recording CSV files in the Edge Impulse ingestion format.
ESP32-Libraries/Kalman.zip	TKJ Electronics Kalman filter library, bundled for reproducibility.
ESP32-Libraries/ei-ilter-akke_1.3.3-arduino-1.0.3-impulse-#1.zip	Edge Impulse Arduino library export, version 1.3.3.
model info/	Edge Impulse model evaluation JSON with confusion matrix and classification report.

Table A.1. AKKE implementation repository structure.

Appendix B: Full Confusion Matrix

The following table reports the full int8 confusion matrix produced by Edge Impulse on the held-out validation set (813 samples, 80/20 split). Rows correspond to the true label; columns correspond to the predicted label. Diagonal entries are correct classifications.

Actual \ Predicted	H1 Stop	H10 MoveL	H2 Listen	H3 Come	H4 Stick	H5 GoGo	H6 SlowD	H7 FallB	H8 Cover	H9 MoveR
Hareket_1 (Stop)	80	0	0	0	0	0	1	0	0	0
Hareket_10 (Move left)	0	69	0	0	0	0	0	0	0	1
Hareket_2 (Listen)	0	0	114	0	0	4	2	1	2	0
Hareket_3 (Come)	0	0	2	72	0	1	0	0	0	0
Hareket_4 (Stick together)	1	0	0	0	60	0	0	0	0	0
Hareket_5 (Go go)	0	0	3	0	0	72	0	0	0	0
Hareket_6 (Slow down)	0	0	0	0	0	0	75	8	1	0
Hareket_7 (Fall back)	0	0	1	0	0	0	7	66	5	0
Hareket_8 (Take cover)	0	0	0	0	0	0	0	3	59	0
Hareket_9 (Move right)	0	3	0	0	0	0	0	0	0	100

Table B.1. Edge Impulse int8 confusion matrix on the 813-sample validation set. Diagonal cells (correct classifications) are highlighted.

Three confusion patterns are visible in Table B.1. First, the most significant confusion occurs within the downward-trajectory gesture group. Slow Down is confused with Fall Back in 8 cases, while Fall Back is confused with Slow Down in 7 cases, producing 15 total cross-errors between these two classes. Fall Back and Take Cover are also confused with each other in 8 total cases: Fall Back is predicted as Take Cover in 5 cases, and Take Cover is predicted as Fall Back in 3 cases. Slow Down is predicted as Take Cover only once, while the reverse direction does not occur. Therefore, the Slow Down / Fall Back / Take Cover cluster accounts for 24 cross-errors in total. Second, Listen is occasionally confused with Go Go, Slow Down, Fall Back, and Take Cover. The largest of these is Listen → Go Go, with 4 cases. Third, the only notable directional confusion outside these groups is Move Right → Move Left, with 3 cases, and Move Left → Move Right, with 1 case. These patterns are consistent with the visual clustering shown in Edge Impulse's Feature Explorer, where some downward-motion and directional-motion gestures partially overlap.